

Spring Boot

Part4

스프링 Controller 클래스 구현

Web 통신 방식의 이해와 요청과 응답에 대한 처리를 담당하는 Controller 클래스

SPRING 첫번째 유인물을 통해 WEB 개발이란 결국 서버와 클라이언트 간 HTML 파일을 요청하고 응답하는 과정을 개발하는 것이라 배웠다.

그리고 요청은 항상 URL을 통해 진행된다는 것도 학습을 통해 알고 있다. 클라이언트가 URL을 통해 특정 HTML 파일을 서버에게 요청하면, 서버는 이 요청에 맞는 HTML 파일을 클라이언트에게 응답하도록 구현하는 것이 바로 웹 개발인 것이다. 결국, 우리가 웹 개발을 학습한다는 건 위에서 언급한 일련의 전 과정을 코드로 구현하는 것이다. 좀 더 자세히 말하자면 웹 개발을 학습한다는 건,

&&html 파일 보내주세요~ /클라이언트에서 url로 요청

- 1) 클라이언트 영역에서 URL을 통해 서버에서 요청을 보내는 방법(HTML, CSS, Javascript, Thymeleaf) 서버는 요청을 html파일로 만들어서 응답 (컨트롤러의 역할 : 자바 스프링)
- 2) 클라이언트로부터 받는 요청을 적절히 처리하고, 처리 결과를 클라이언트에게 HTML로 응답하는 서버를 만드는 방법(Spring, Database)

이 두 가지를 배운다는 것이다.

먼저 클라이언트로부터 들어온 요청을 받아 응답을 처리할 수 있도록 서버를 구축해야한다. 여기서 말하는 서버는 Spring 프로젝트를 의미한다.

Spring 프로젝트에서 URL을 통해 들어온 요청을 받고, 받은 요청에 대해 HTML로 응답하는 기능을 가진 클래스를 Controller라고 부른다. 그래서 우선, Controller 역할을 하는 클래스부터 만들어보자. Controller 클래스가 되기 위해서는 반드시 아래의 내용을 지켜야 한다.

- 1) 컨트롤러 역할의 클래스는 반드시 default 패키지에 위치해야 한다. 여기서 말하는 default 패키지란 Spring 프로젝트 생성 시 기본적으로 존재하는 패키지를 말한다.
- 2) 생성한 파일의 class 선언문 위에 @Controller 어노테이션을 추가한다.(자바에서 @는 Annotation[어노테이션]이라 읽는다)

위 두 가지만 지키면 Controller 클래스가 만들어진다. 참고로, Spring 프로젝트에는 Controller 역할의 클래스는 여러 개 생성할 수 있다.

@Controller 어노테이션은 해당 클래스의 객체 생성을 스프링 컨테이너에게 지시하며, controller 역할의 클래스임을 인지시키는 기능을 하는 어노테이션이다.

지금까지의 설명을 토대로 만든 Controller 클래스는 아래와 같다.

```
package com.green.second.controller;

import org.springframework.stereotype.Controller;

@Controller
public class TestController {

}
```

- 해당 프로젝트의 default패키지는 com.green.second 다. 그리고 TestController 클래스는 default 패키지 안에 만들어져 있다.
- 클래스 선언문 위의 @Controller 어노테이션을 추가하였다.
- 위 두 조건이 만족한다면 스프링 컨테이너는 TestController를 Controller로 인지하며, 스프링 프로젝트가 실행되면 TestController 클래스에 대한 객체를 생성해준다(IOC, DI)
(프로젝트가 실행되면 TestController testController = new TestController() 코드 실행!)

이렇게 만들어진 Controller 클래스는 URL로 전달되는 요청을 받고, 요청에 대한 응답을 할 수 있다. 그리고 이러한 기능은 Controller 클래스 안의 작성된 메서드를 통해 이루어진다. 다음 슬라이드에서 내용을 살펴보자.

```
@Controller
public class TestController {

    @GetMapping("/a")
    public String test1(){
        return "member_list";
    }

    @GetMapping("/b")
    public String test2(){
        return "member_form";
    }

    @PostMapping("/c")
    public String test3(){
        return "member_detail";
    }
}
```

- Controller 클래스에는 현재 3개의 메서드를 선언했다. 각각의 메서드는 요청에 대한 응답을 처리한다.
 - 결국, 현재 코드는 3가지 요청에 대한 각각의 응답을 코드로 구현한 것이다.
 - 메서드 위에는 `@GetMapping()` 또는 `@PostMapping()` 어노테이션이 작성되었다. 해당 어노테이션은 요청 URL 정보를 갖고 있다.
 - 메서드의 리턴에는 요청에 대한 응답 HTML파일명을 작성한다. HTML 파일명을 작성할 때에는 '.html' 확장자는 제외한다. 또한, 응답을 위한 HTML 파일은 반드시 `resources/templates` 폴더 안에 위치해야 한다. `return`에 작성한 것은 응답할 HTML 파일명이므로, 메서드의 리턴타입은 무조건 String이다.
-
- localhost:8080/a 요청에 대한 응답 메서드이다.
 - localhost:8080/a 요청이 오면 `resources/templates/member_list.html` 파일을 응답한다.
 - localhost:8080/b 요청에 대한 응답 메서드이다.
 - localhost:8080/b 요청이 오면 `resources/templates/member_form.html` 파일을 응답한다.
 - localhost:8080/c 요청에 대한 응답 메서드이다.
 - localhost:8080/c 요청이 오면 `resources/templates/member_detail.html` 파일을 응답한다.

자바 학습을 통해 메서드를 실행하기 위해서는 반드시 메서드 정의 후 메서드 호출을 해야 한다는 것을 배웠다. 하지만 Controller 클래스에서는 메서드를 정의만 하였으며, 정의한 메서드를 호출하는 코드는 어디에도 작성하지 않았다. 그럼에도 불구하고 메서드가 실행되는 이유는 스프링 컨테이너가 요청 URL에 맞는 응답 메서드를 자동으로 호출해주기 때문이다.

컨트롤러 클래스 안에 선언한 메서드는 URL로 전달되는 요청을 받아 처리한다. 다음 이야기를 하기 전에 우선 요청 URL을 보내는 4가지 방향에 대해 알아보겠다.

그리고 다시 말하지만 요청은 클라이언트(HTML 영역)가 서버에게 보내는 것이다. 서버로 요청을 보내는 방식은 총 네 가지가 있다.

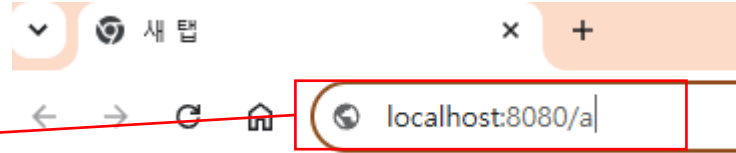
- 1) 웹 브라우저의 주소 입력란에 URL을 직접 입력한다. 1), 2), 4) --> @GetMapping
- 2) HTML a 태그 href 속성에 URL을 입력한다.
- 3) HTML form 태그 action 속성에 URL을 입력한다. --> @PostMapping
- 4) Javascript의 location.href 명령어에 URL을 입력한다.

다음 슬라이드에서 하나씩 예시를 살펴보자.

```
@Controller
public class TestController {

    @GetMapping("/a")
    public String test1(){
        return "member_list";
    }
}
```

웹 브라우저의 주소 입력란에 URL을 직접 입력



웹 브라우저에 localhost:8080/a 입력으로 요청하면
test1메서드가 요청을 받아, 요청에 대한 응답으로
resources/templates/member_list.html 파일을 실행
웹 브라우저에 url을 직접 입력할때만 localhost:8080을 붙인다.

```
@GetMapping("/b")
public String test2(){
    return "member_form";
}
```

HTML a 태그 href 속성에 URL을 입력

`<a href="/b">페이지 이동</a>`

a 태그의 href 속성에 /b 입력으로 요청하면
test2메서드가 요청을 받아, 요청에 대한 응답으로
resources/templates/member_form.html 파일을 실행

```
@PostMapping("/c")
public String test3(){
    return "member_detail";
}
```

HTML form 태그 action 속성에 URL을 입력

`<form action="/c" method="post">`
`<button type="submit">전송</button>`
`</form>`

form 태그의 action 속성에 /a 입력으로 요청하면
test3메서드가 요청을 받아, 요청에 대한 응답으로
resources/templates/member_detail.html 파일을 실행
form 태그 요청은 @PostMapping() 어노테이션을 사용한다.

```
@GetMapping("/d")
public String test4(){
    return "member_detail";
}
```

Javascript의 location.href 명령어에 URL을 입력

`<script>`
`location.href = '/d';`
`</script>`

location.href 코드에 /d 입력으로 요청하면
test4메서드가 요청을 받아, 요청에 대한 응답으로
resources/templates/member_detail.html 파일을 실행

URL로 요청을 보낼 때 필요하다면 데이터도 함께 전달 할 수 있다. URL로 데이터를 전달하는 방식은 아래와 같이 크게 두 가지 방식이 있다.

URL Parameter 방식

- 요청 URL에 데이터를 포함하는 구조다.
- 주로 전달할 데이터가 1개일 때 사용한다.
- ex> localhost:5173/items/1, localhost:5173/items/2

Query String 방식

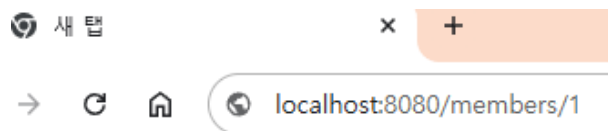
- url 뒤에 전달할 데이터를 'key=value' 형태로 작성하며, url과 전달 데이터를 '?'로 구분하여 url을 작성하는 방식이다.
- react를 사용하지 않을 때 많이 사용하는 방식이며, 전달할 데이터가 2개 이상일 때 URL Parameter 방식 대신 사용한다.

ex>

- localhost:5173/items?itemCode=1
-> '/items' url로 이동하면서 itemCode라는 이름으로 1을 전달하겠다는 의미이다.
- localhost:5173/items?itemCode=1&itemName=상의
-> '/items' url로 이동하면서 itemCode라는 이름으로 1을, itemName이라는 이름으로 '상의' 라는 2개의 데이터를 가져가겠다는 의미이다.
전달한 데이터가 여러개일 경우, 위의 예시처럼 '&'로 연결한다.

아래 코드는 요청을 보내는 세 가지 방식(URL 직접 입력, a 태그, location.href)에 데이터를 실어보내는 예시이다. form 태그로 보내지는 요청은 자동으로 데이터를 가져가기 때문에 별도로 추가하는 코드가 없다.

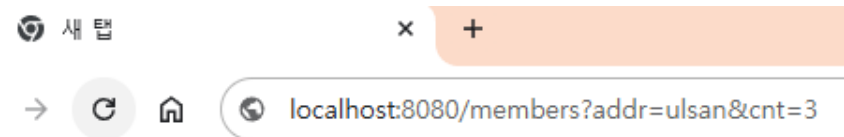
URL Parameter 방식



```
<a href="/members/1">페이지 이동</a>
```

```
location.href = '/members/3';
```

Query String 방식



```
<a href="/members?addr=ulsan&cnt=3">페이지 이동</a>
```

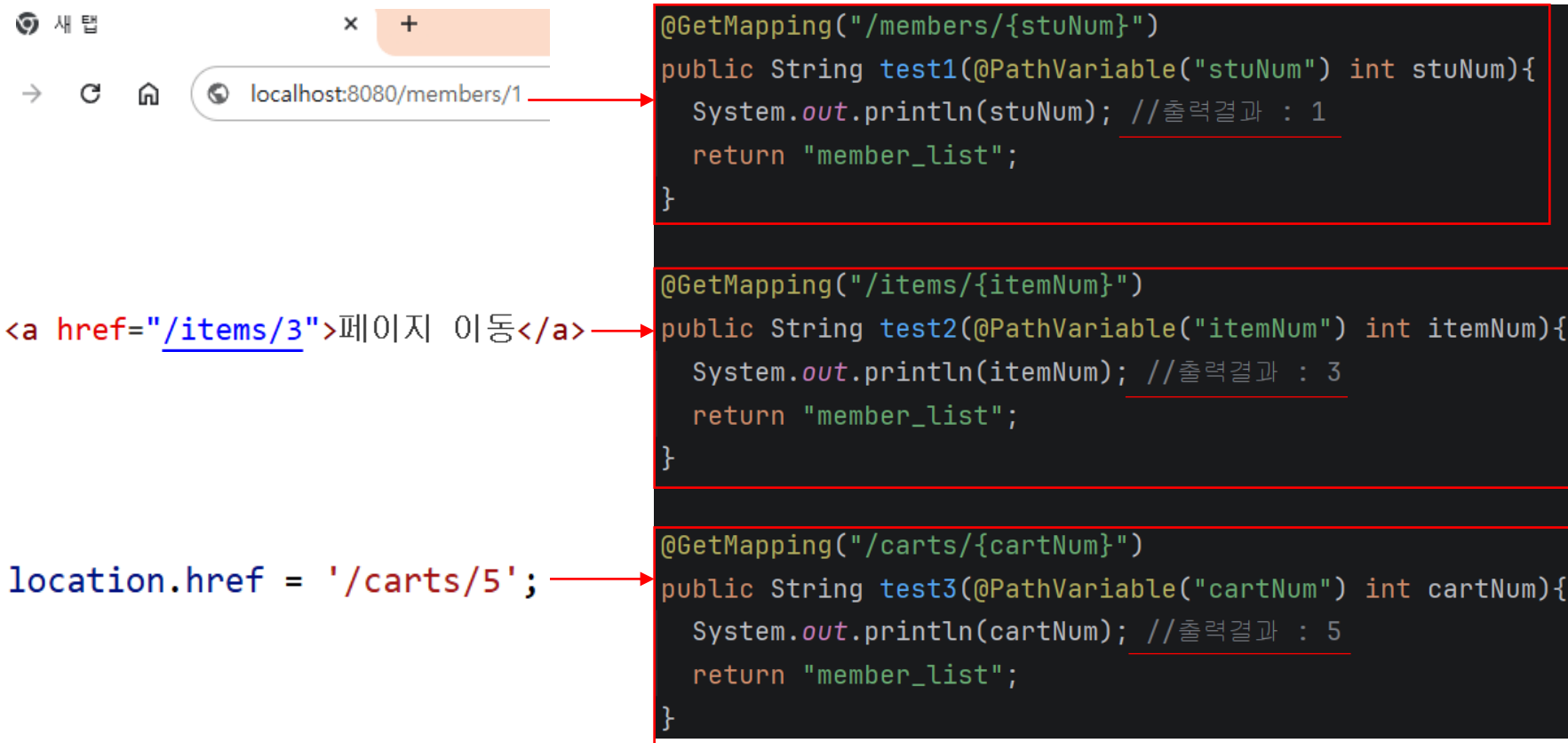
```
location.href = '/members?addr=ulsan&cnt=3';
```

위 예시를 보면 알겠지만 웹 브라우저에 직접 입력하는 방식, a태그 사용 방식, location.href를 사용한 방식 모두 다른 점은 없다!

그렇다면, URL에 데이터를 포함하여 요청이 들어올 때, 컨트롤러에서는 어떻게 요청과 함께 전달되는 데이터를 받을 수 있을까.

먼저 URL Parameter 방식의 요청으로 전달되는 데이터는 아래와 같이 받는다.

URL Parameter 방식으로 전달된 데이터 받기



The image shows a browser window with the address bar at `localhost:8080/members/1`. A red arrow points from the `1` in the URL to the `test1` method in the code. Below the browser, there are two more examples. The first shows an HTML anchor tag `<a href="/items/3">페이지 이동</a>` with a red arrow pointing to the `test2` method. The second shows a Java code snippet `location.href = '/cars/5';` with a red arrow pointing to the `test3` method. The code snippets are as follows:

```
@GetMapping("/members/{stuNum}")
public String test1(@PathVariable("stuNum") int stuNum){
    System.out.println(stuNum); //출력결과 : 1
    return "member_list";
}

@GetMapping("/items/{itemNum}")
public String test2(@PathVariable("itemNum") int itemNum){
    System.out.println(itemNum); //출력결과 : 3
    return "member_list";
}

@GetMapping("/cars/{cartNum}")
public String test3(@PathVariable("cartNum") int cartNum){
    System.out.println(cartNum); //출력결과 : 5
    return "member_list";
}
```

URL로 전달되는 데이터를 받을 때는 {}를 사용한다.

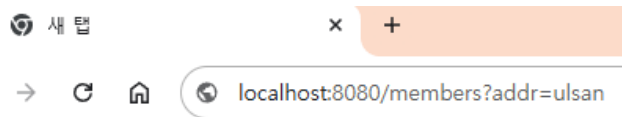
{ }안에 작성된 내용을 데이터를 받을 변수명이라 생각하면 된다.

URL Parameter로 전달되는 데이터는 메서드에 `@PathVariable` 어노테이션을 사용하여 받는다.

다음으로 Query String 방식으로 전달되는 데이터를 받아 코드를 알아보자.

Query String 방식으로 전달되는 데이터를 받은 방식은 @RequestParam 어노테이션을 이용하는 방법과 커맨드 객체를 이용하는 방법이 있다.

먼저, @RequestParam 어노테이션을 이용한 방식이다. 참고로 form 태그로 전달되는 데이터도 동일하게 @RequestParam 어노테이션으로 받는다.



```
@GetMapping("/members")
public String test1(@RequestParam(name = "addr") String addr){
    System.out.println(addr); //출력결과 : ulsan
    return "member_list";
}
```


location.href = '/items?itemNum=1&itemName=pencil';

```
@GetMapping("/items")
public String test2(@RequestParam(name="itemNum") int itemNum,
    @RequestParam(name = "itemName") String itemName){
    System.out.println(itemNum); //출력결과 : 1
    System.out.println(itemName); //출력결과 : pencil
    return "member_list";
}
```

URL Parameter 방식은 ? 이전까지가 요청 URL이며, ? 이후에는 전달되는 데이터를 표현한다.

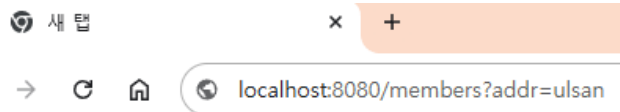
Query String 방식으로 전달되는 데이터를 커맨드 객체를 이용하여 데이터를 받는 방식을 알아보자.

커맨드 객체는 컨트롤러 클래스에 선언한 메서드의 매개변수로 전달되는 객체를 의미한다. 데이터를 전달받기 위한 커맨드 객체는 반드시 전달되는 데이터와 동일한 이름의 변수와 getter, setter가 존재해야 한다. form 태그로 전달되는 데이터도 커맨드 객체를 활용하여 데이터를 받을 수 있다.

```
@Getter
@Setter
@ToString
public class TestDTO {
    private String addr;
    private int itemNum;
    private String itemName;
}
```

TestDTO는 요청 시 전달되는 데이터를 받기 위해 만든 클래스다.

전달되는 데이터의 이름과 동일한 변수를 가지고 있으며 getter와 setter 또한 가지고 있다.



```
@GetMapping("/members")
public String test1(TestDTO testDTO){
    //출력결과-> addr : ulsan, itemNum : 0, itemName : null
    System.out.println(testDTO);
    return "member_list";
}
```


location.href = '/items?itemNum=1&itemName=pencil';

```
@GetMapping("/items")
public String test2(TestDTO testDTO){
    //출력결과-> addr : null, itemNum : 1, itemName : pencil
    System.out.println(testDTO);
    return "member_list";
}
```

지금까지 요청을 보내는 네 가지 방식 중 URL 직접 입력, a 태그, location.href 등 세 가지 방식으로 전달되는 데이터를 받는 코드를 알아보았다.

이 세가지 방식은 URL Parameter 방식으로 데이터가 전달되는지, Query String 방식으로 데이터가 전달되는지에 따라 컨트롤러에서 데이터를 받는 방식이 달랐다.

이번에는 **form 태그**로 들어오는 요청에서 전달되는 데이터를 받는 방법을 알아보자. form 태그는 주로 form 요소로부터 사용자가 입력한 데이터를 전달받을 때 사용한다.

아래는 form 태그의 사용 예시와 설명이다.

```
<form action="/r2" method="post">
  이름 : <input type="text" name="name"> <br>
  나이 : <input type="text" name="age"> <br>
  전공
  <select name="major">
    <option value="컴퓨터공학">컴 공</option>
    <option value="정보통신공학">정보통신</option>
    <option value="전기전자과">전자과</option>
  </select> <br>
  성별
  <input type="radio" name="gender" value="male" checked> 남
  <input type="radio" name="gender" value="female" checked> 여
  <button type="submit">전송</button>
</form>
```

- form요소에 입력한 데이터를 spring으로 전달하기 위해서는 form요소를 form태그에 감싼다.
- 데이터 전송을 위한 버튼은 type 속성값을 submit으로 지정한다.
- submit 버튼도 반드시 form 태그 안에 있어야 함.
- form태그의 action속성에 요청URL을 입력한다.
- form태그의 method 속성값은 반드시 post로 지정한다.
- 이렇게 구성하면 submit 버튼 클릭 시 form요소에 입력한 데이터(value 속성 값)를 가지고 요청url로 접근한다.
- form태그로 전송되는 요청을 받기위해 Spring의 메서드는 @PostMapping을 사용한다.
- form요소를 구분하기 위해서 form 요소 각각에 name 속성을 부여한다.

form 태그의 method="post"를 작성하면 요청 URL에 전달되는 데이터가 보이지 않는다. method="get"을 작성하거나, a태그, location.href를 이용하여 데이터를 전달하면 전달 데이터가 URL에 보인다. 그렇기 때문에 민감한 데이터의 경우에는 반드시 form태그와 함께 method="post"를 작성하여 데이터를 전달한다.

form 태그로 전달되는 데이터는 컨트롤러에서 @RequestParam 어노테이션을 이용하여 받거나, 커맨드 객체를 이용하여 받는다.

아래 코드는 form 요소로 전달되는 데이터를 @RequestParam 어노테이션으로 받는 코드이다.

```
<form action="/r2" method="post">
  이름 : <input type="text" name="name"> <br>
  나이 : <input type="text" name="age"> <br>
  전공
  <select name="major">
    <option value="컴퓨터공학">컴 공</option>
    <option value="정보통신공학">정보통신</option>
    <option value="전기전자과">전자과</option>
  </select> <br>
  성별
  <input type="radio" name="gender" value="male" checked> 남
  <input type="radio" name="gender" value="female" checked> 여
  <button type="submit">전 송</button>
</form>
```

```
@PostMapping("/r2")
public String test5(
    @RequestParam(name = "name") String name,
    @RequestParam(name = "age") int age,
    @RequestParam(name = "major") String major,
    @RequestParam(name = "gender") String gender){

    return "test";
}
```

form 태그의 action="/r2" method="post" 속성에 맞춰 요청을 받을 수 있게 메서드에서는 @PostMapping("/r2")를 작성하였다.

form 태그에서 전달되는 form 요소의 name 속성 값과 동일하게 @RequestParam 어노테이션을 이용하여 데이터를 받는다.

다시 말하지만 form 태그 안에 위치한 form 요소의 value값을 받아온다.

아래 코드는 form 요소로 전달되는 데이터를 커맨드객체로 받는 코드이다.

```
@Getter @Setter @ToString
public class TestDTO {
    private String name;
    private int age;
    private String major;
    private String gender;
}
```

왼쪽과 같이 form 요소로 넘어오는 데이터를 받을 수 있는 클래스자료형을 만들었다.

이때, 멤버변수의 이름과 form 요소들의 name 속성이 반드시 일치해야 한다.

또한, 반드시 getter와 setter가 존재해야 한다.

```
<form action="/r2" method="post">
    이름 : <input type="text" name="name"> <br>
    나이 : <input type="text" name="age"> <br>
    전공
    <select name="major">
        <option value="컴퓨터공학">컴 공</option>
        <option value="정보통신공학">정 보 통신</option>
        <option value="전기전자과">전 자 과</option>
    </select> <br>
    성별
    <input type="radio" name="gender" value="male" checked> 남
    <input type="radio" name="gender" value="female" checked> 여
    <button type="submit">전 송</button>
</form>
```

```
@PostMapping("/r2")
public String test5(TestDTO testDTO){
    //전달받은 데이터 확인을 위해 toString() 호출
    System.out.println(testDTO);
    return "test";
}
```

form 태그로 전달되는 데이터는 커맨드 객체인 testDTO에 자동으로 들어온다. testDTO 객체를 출력해보면 전달받은 데이터를 확인할 수 있다.

지금까지 요청을 통해 전달되는 데이터를 컨트롤러의 메서드에서 받는 방법을 알아보았다. 이번에는 컨트롤러 메서드에서 응답하는 **html 파일로 데이터를 전달하는 방법**을 알아보자. 다행히 메서드에서 응답 html로 데이터를 전달하는 방법은 한 가지 뿐이다. html로 전달할 데이터가 존재하는 메서드의 매개변수에 Model 인터페이스 객체만 선언하면 된다. 아래 코드를 보자.

```
@PostMapping("/r2")
public String test5(Model model){
    model.addAttribute("name", "kim");
    model.addAttribute("age", 20);
    model.addAttribute("student", new StudentVO());
    return "test";
}
```

- 위 메서드를 보면 매개변수 자리에 Model 인터페이스 객체가 선언되어 있다.
- **Model 인터페이스 객체의 addAttribute() 메서드를 사용하면 응답 html에 데이터를 전달할 수 있다.**
- addAttribute 메서드의 첫번째 매개변수는 '전달할 데이터 이름', 두번째 매개변수는 전달할 데이터를 의미한다.
- model.addAttribute("name", "kim"); 이라는 코드를 해석하면 name 이라는 이름으로 "kim"이라는 문자열 데이터를 전달하겠다는 코드이다.
- 이렇게 전달된 데이터는 **html에서 "name"으로 받을 수 있으며, 받은 데이터는 "kim"이라는 문자열이 된다.**
- addAttribute 메서드의 두번째 매개변수 자료형은 Object다. 즉, 두번째 매개변수로는 모든 자료형 데이터를 넘길 수 있다는 의미이다.
- 응답 html 파일로 전달할 데이터가 여러 개라면 addAttribute 메서드를 여러 번 호출할 수 있다.
- 커맨드 객체의 경우 addAttribute 메서드를 이용하여 데이터를 응답 html로 넘기지 않아도 자동으로 넘어간다. 이때, 커맨드 객체는 **클래스명에서 앞글자만 소문자**로 바꾼 이름으로 응답 html로 전달된다.